Pypomp: Inference for partially observed Markov process models in Python with JAX

Aaron J. Abkemeier[1] AARONABK@UMICH.EDU
Jun Chen[2] CHENJUNC@UMICH.EDU
Kevin Tan[3] KEVTAN@WHARTON.UPENN.EDU
Jesse Wheeler[4] JESSE.WHEELER@USU.EDU
Bo Yang[5] YBB@UMICH.EDU
Kunyang He[1] KUNYANGHE@UMICH.EDU
Jonathan Terhorst[1] JONTH@UMICH.EDU
Aaron A. King[6] KINGAA@UMICH.EDU
Giles Hooker[3] GHOOKER@WHARTON.UPENN.EDU
Edward L. Ionides[1] IONIDES@UMICH.EDU

[1] *University of Michigan, Department of Statistics;* [2] *Duke University, Department of Statistics;* [3] *University of Pennsylvania, Department of Statistics;* [4] *Utah State University, Department of Mathematics & Statistics;* [5] *Columbia University, Department of Marketing;* [6] *University of Michigan, Department of Ecology & Evolutionary Biology*

Editor:

Abstract

Nonlinear, non-Gaussian partially observed Markov process (POMP) models arise in various areas including epidemiology, ecology, and robotics. Model-based inferences typically involve computationally intensive Monte Carlo methods. We introduce Pypomp, a Python library for data analysis via POMP models that uses JAX to take advantage of GPU hardware, automatic differentiation, and just-in-time compilation. By design, Pypomp supports model development and data analysis as well as method development, following the approach of the pomp R package. However, Pypomp obtains considerably higher performance: Pypomp running on one GPU can obtain the throughput of 8,000 CPU cores running pomp. A central algorithm for Pypomp is the particle filter, and Pypomp is faster on our benchmark tests than other GPU-compatible particle filter libraries. The particle filter is differentiable, with a tunable bias-variance tradeoff.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially observed Markov process (POMP) models provide a flexible framework for modeling dynamic systems whose underlying state is not directly observed. POMP models are also known as state-space models or hidden Markov models. Pypomp is a Python library combining advances in hardware, software, and algorithms for POMP models, with an emphasis on inference via the particle filter (PF). PF, also known as sequential Monte Carlo, provides an estimate of the likelihood of a POMP model (Arulampalam et al., 2002). The basic PF algorithm needs to simulate from the process model but does not evaluate

its transition probabilities, a property known as plug-and-play (P&P) (Bretó et al., 2009). Pypomp focuses on P&P methods because they facilitate scientific modeling, enabling applications in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al., 2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), robotics (Jonschkowski et al., 2018) and other domains where nonlinear mechanistic stochastic dynamic models are pertinent. Pypomp provides P&P inference methods extending PF: maximum likelihood parameter estimation by iterated filtering (IF2) (Ionides et al., 2015), Bayesian inference by particle Markov chain Monte Carlo (Andrieu et al., 2010), and stochastic gradient descent (SGD) via a differentiable particle filter (DPF) algorithm called a measurement-off-parameter (MOP) particle filter (Tan et al., 2026).

Pypomp provides a framework for specifying models and inference methods, a range of pre-built models and methods, and various graphical and diagnostic tools to assist practical data analysis. Thus, Pypomp supports complete data analysis workflows, facilitating the evaluation of a range of models and statistical methods. Models and methods share an interface so that new models can readily be tested on pre-existing methods, and new methods can be tested on pre-existing models. These design features are inherited from the R package pomp (King et al., 2016). Although the method interface of Pypomp (and pomp) is not limited to P&P algorithms, experience has shown that P&P approaches are convenient for scientific data analysis. Theoretical obstacles have limited the practical scope of non-P&P algorithms (Roberts and Stramer, 2001; Karppinen et al., 2024).

Pypomp leverages JAX (Bradbury et al., 2018) to provide just-in-time (JIT) compilation, automatic differentiation (AD), and capability to run on massively parallel hardware such as GPUs. When the state space is small and discrete, or the model is linear Gaussian or close enough for effective approximation using Gaussian-inspired filters such as the extended Kalman filter, the particle filter is unnecessary, and the Dynamax library (Linderman et al., 2025) provides JAX-based solutions. Other JAX-based libraries providing PF and DPF methods for POMP models include PFJAX (Lysy et al., 2022), Cuthbert (Duffield, 2026) and pypfilt (Moss, 2024). PyDPF (Brady et al., 2026) provides a DPF using Pytorch and focused on POMP models parameterized using neural networks. The Python library particles (Chopin and Papaspiliopoulos, 2020) provides a general framework for sequential Monte Carlo computation without AD. Pypomp distinguishes itself from these other packages by (i) following the design decisions of pomp that support a full data analysis workflow; (ii) superior performance on benchmark tests. Pypomp is not limited to PF methods, for example, the approximate Bayesian computing (ABC) P&P algorithm of Marjoram et al. (2003) has been implemented. However, PF has proved an effective tool for the development and analysis of POMP models in scientific contexts.

With enough particles and method replications to saturate it, one GPU runs iterated filtering as fast as 3,310 to 8,341 CPU cores running pomp (Appendix C). Such replications are routine in practice: practical likelihood maximization involves searching from many initial parameter vectors, and Monte Carlo adjusted profile (MCAP) confidence intervals (Ionides et al., 2017), implemented in Pypomp as `mcap`, require maximizations. Profile likelihood have been found preferable to Fisher information for confidence interval construction for linear Gaussian POMP models (Wheeler and Ionides, 2025). For Monte Carlo methods, MCAP algorithms can lead to efficient inference even in a limit where the Monte Carlo error grows as the amount of data increases (Ning et al., 2021).

Algorithms to take advantage of JIT, AD, and GPU are an area of active development. Pypomp implements the MOP P&P DPF algorithm of Tan et al. (2026), with the resulting derivative being called DMOP and implemented as the `dmop` method. DMOP is competitive on benchmark problems when combined with Adam SGD, especially when using IF2 for a warm start (Tan et al., 2026). Pypomp provides inverse transform sampling methods for use in `rproc` and `rmeas` since the rejection sampling used in some `jax.random` samplers is inefficient on GPUs (Giles, 2016; Giles and Beentjes, 2024; Temme, 1992). These Pypomp samplers run 23 to 44 times faster than their `jax.random` counterparts on a GPU. Quantiles of draws from inverse sampling and rejection sampling differ by at most 1.1% of a standard deviation on average (Appendix D).

2 POMP Modeling in Pypomp

A POMP model consists of an unobserved Markov process $\{X(t), t_0 \leq t \leq t_N\}$ and observable $Y_{1:N}$ at times $t_{1:N}$ with observed value $y^*_{1:N}$. The initial state is $X(t_0)$, $X_n := X(t_n)$, and Y_n is independent of Y_m, X_m, $m \neq n$, conditional on X_n. P&P methods need only a simulator for the initial state, $X_0 \sim f_{X_0}(x_0;\theta)$, a simulator for the state transition, $X_{n+1} \sim f_{X_{n+1}|X_n}(x_{n+1}|x_n;\theta)$, and the measurement density $f_{Y_n|X_n}(y_n|x_n;\theta)$, where θ denotes the model parameters. The user passes data and JAX-compatible code for these functions to build a `Pomp` class model object using the `Pomp` consctructor function. For P&P methods, it suffices to specify an initial value for the latent dynamic system (known as `rinit`), a latent process simulator (`rproc`), a measurement density simulator (`rmeas`) and a measurement density evaluator (`dmeas`). Tutorials are available at `https://github.com/pypomp/tutorials`. Models included in Pypomp include a model for historical monthly cholera mortality in Dhaka, Bangladesh (`pypomp.models.dacca`), historical weekly measles cases in London, UK (`pypomp.models.measles.pomp`), daily closing price for the S&P 500 index (`pypomp.models.spx`), and a panel model for measles cases in up to 1422 urban and rural districts in England and Wales (`pypomp.models.measles.panel_pomp`).

Below, we demonstrate maximum likelihood estimation with IF2 (`mif`) followed by refinement using SGD (`train`) and then log-likelihood evaluation using PF (`pfilter`), on the `spx` model. Each method stores its convergence records, execution time, and arguments in the `results_history` attribute of the `Pomp` object, enabling sequential method chaining and diagnostic investigation by helper methods such as `traces`.

```
import pypomp as pp, jax
spx_obj = pp.models.spx()
spx_obj.mif(J=1000, M=10, key=jax.random.key(1),
    rw_sd = pp.RWSigma(
        sigmas={"mu": 0.02, "kappa": 0.02, "theta": 0.02,
            "xi": 0.02, "rho": 0.02, "V_0": 0.1},
        init_names=["V_0"],
    ).geometric_cooling(a=0.5))
spx_obj.train(J=1000, M=10, optimizer=pp.Adam(),
    eta = pp.LearningRate(
        {name: 0.001 for name in spx_obj.canonical_param_names}
```

Table 1: Wall-clock time in seconds for 36 IF2 searches. Ratio is pomp time divided by Pypomp time. CPU cores matched is Ratio × 36. Peak cores matched is the highest value of CPU cores matched in GPU scaling tests with additional particles (see Appendix C).

Model	`pomp` (36 cores)	Pypomp (1 GPU)	Ratio	CPU cores matched	Peak cores matched
Cholera, Dhaka	1,609	28.3	57×	2,045	3,310
Measles, London	2,923	32.2	91×	3,264	5,432
Measles, 4-town panel	8,262	98.2	84×	3,030	3,970
S&P 500	643	34.8	19×	666	8,341

```
    ).geometric_decay(decay_rate=0.5, M=10))
spx_obj.pfilter(J=1000, reps=10)
```

A collection of POMP models is a PanelPOMP (Bretó et al., 2020), and each constituent POMP is called a *unit*. Parameters may be unit-specific or shared between units. Marginalized panel iterated filtering (MPIF) (Wheeler et al., 2025) and DMOP provide likelihood maximization for models of class `PanelPomp`.

3 Performance

Two performance questions are: (i) what is the benefit of using GPU hardware versus CPU? (ii) how does Pypomp compare in speed to altnernative GPU-based particle filter libraries? To address (i), we compare Pypomp on a GPU with pomp on a CPU because that informs a natural decision for scientists considering Pypomp as a data analysis tool. Model code in pomp is usually written via compiled snippets of C code, and so pomp has comparable speed to other high-performance CPU-based packages for POMP models (FitzJohn et al., 2020). Results are shown in Table 1 on four models with different characteristics. Additional details are in Appendix A and `https://github.com/pypomp/quant`. Relative efficiency varies with the number of particles, with the highest efficiency occurring when the GPU cores are saturated (Appendix C). The GPU times for the measles models use the GPU-adapted samplers in `pypomp.random`: with standard algorithms, such as `jax.random` samplers, IF2 for the London measles model took 46 times longer (Appendix D). To address (ii), Appendix B runs the same particle filter workloads in five other Python libraries. On the London measles model, `pfjax` and `cuthbert` took 23 to 26 times as long as Pypomp on a GPU.

4 Governance and Acknowledgments

The source code for Pypomp is openly available at `https://github.com/pypomp/pypomp`. The project is run by the Pypomp organization (`https://github.com/pypomp`) under democratic governance rules that welcome new developers and community contributions. The Pypomp project is supported by National Science Foundation grants OAC-2607869 and OAC-2607870. The developers acknowledge coding assistance from artificial intelligence models by OpenAI, Anthropic and Google.

References

Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(3):269–342, 2010. doi: 10.1111/j.1467-9868.2009.00736.x.

M. Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, 2002. doi: 10.1109/78.978374.

Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470.

Alexander D. Becker and Bryan T. Grenfell. tsiR: An R package for time-series Susceptible-Infected-Recovered models of epidemics. *PLOS ONE*, 12(9):e0185528, 2017. doi: 10.1371/journal.pone.0185528.

Julie C. Blackwood, Daniel G. Streicker, Sonia Altizer, and Pejman Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110 (51):20837–20842, 2013. doi: 10.1073/pnas.1308817110.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, ake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.

John-Joseph Brady, Benjamin Cox, Yunpeng Li, and Víctor Elvira. PyDPF: A Python package for differentiable particle filtering, 2026.

Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003.

Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, 2009. doi: 10.1214/08-AOAS201.

Carles Bretó, Edward L. Ionides, and Aaron A. King. Panel Data Analysis via Mechanistic Models. *Journal of the American Statistical Association*, 115(531):1178–1188, 2020. doi: 10.1080/01621459.2019.1604367.

Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer, 2020. doi: 10.1007/978-3-030-47845-2.

Sam Duffield. cuthbert: State-space model inference with JAX, 2026. URL `https://pypi.org/project/cuthbert/`.

Richard G. FitzJohn, Edward S. Knock, Lilith K. Whittles, Pablo N. Perez-Guzman, Sangeeta Bhatia, Fernando Guntoro, Oliver J. Watson, Charles Whittaker, Neil M. Ferguson, Anne Cori, Marc Baguelin, and John A. Lees. Reproducible parallel inference and simulation of stochastic state space models using odin, dust, and mcstate. *Wellcome Open Research*, 5, 2020. doi: 10.12688/wellcomeopenres.16466.2.

Spencer J. Fox, Michael Lachmann, Mauricio Tec, Remy Pasco, Spencer Woody, Zhanwei Du, Xutong Wang, Tanyi A. Ingle, E. Javan, Maytal Dahan, Kelly Gaither, Mark E. Escott, Stephen I. Adler, S. Claiborne Johnston, James G. Scott, and Lauren A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119.

Michael B. Giles. Algorithm 955: Approximation of the Inverse Poisson Cumulative Distribution Function. *ACM Transactions on Mathematical Software*, 42(1):1–22, 2016. doi: 10.1145/2699466.

Michael B. Giles and Casper Beentjes. Approximation of an inverse of the incomplete Beta Function. In Kevin Buzzard, Alicia Dickenstein, Bettina Eick, Anton Leykin, and Yue Ren, editors, *Mathematical Software – ICMS 2024*, volume 14749, pages 207–214. Springer Nature Switzerland, Cham, 2024. doi: 10.1007/978-3-031-64529-7_22. Series Title: Lecture Notes in Computer Science.

Daihai He, Edward L. Ionides, and Aaron A. King. Plug-and-play inference for disease dynamics: Measles in large and small populations as a case study. *Journal of The Royal Society Interface*, 7(43):271–283, 2010. doi: 10.1098/rsif.2009.0151.

Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, 2015. doi: 10.1073/pnas.1410597112.

Edward L. Ionides, Carles Breto, Joonha Park, R. Alexander Smith, and Aaron A. King. Monte Carlo profile confidence intervals for dynamic systems. *Journal of The Royal Society Interface*, 14(132):20170126, 2017. doi: 10.1098/rsif.2017.0126.

Rico Jonschkowski, Divyam Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors. *Robotics: Science and Systems XIV*, 2018. doi: 10.48550/arXiv.1805.11122.

Santeri Karppinen, Sumeetpal S Singh, and Matti Vihola. Conditional particle filters with bridge backward sampling. *Journal of Computational and Graphical Statistics*, 33(2): 364–378, 2024. doi: 10.1080/10618600.2023.2231514.

Aaron A. King, Edward L. Ionides, Mercedes Pascual, and Menno J. Bouma. Inapparent infections and cholera dynamics. *Nature*, 454(7206):877–880, 2008. doi: 10.1038/nature07084.

Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical Inference for Partially Observed Markov Processes via the *R* Package **pomp**. *Journal of Statistical Software*, 69(12), 2016. doi: 10.18637/jss.v069.i12.

Scott W. Linderman, Peter Chang, Giles Harper-Donnelly, Aleyna Kara, Xinglong Li, Gerardo Duran-Martin, and Kevin Murphy. Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, 10(108):7069, 2025. doi: 10.21105/joss.07069.

Martin Lysy, Pranav Subramani, Jonathan Ramkissoon, Mohan Wu, Michelle Ko, and Kanika Choptra. Pfjax: Particle filtering in JAX, 2022. URL `https://github.com/mlysy/pfjax`.

John A. Jr. Marino, Scott D. Peacor, David B. Bunnell, Henry A. Vanderploeg, Steven A. Pothoven, Ashley K. Elgin, James R. Bence, Jing Jiao, and Edward L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, 2019. doi: 10.1002/ecy.2583.

Paul Marjoram, John Molitor, Vincent Plagnol, and Simon Tavaré. Markov chain Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences*, 100(26): 15324–15328, 2003. doi: 10.1073/pnas.0306899100.

Matthew S. Mietchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating SARS-CoV-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, 2024.

Robert Moss. pypfilt: A particle filter for Python. *Journal of Open Source Software*, 9(96): 6276, 2024. doi: https://doi.org/10.21105/joss.06276.

N. Ning, E. L. Ionides, and Y. Ritov. Scalable Monte Carlo inference and rescaled local asymptotic normality. *Bernoulli*, 27:2532–2555, 2021. doi: 10.3150/20-BEJ1321.

G. O. Roberts and O. Stramer. On inference for partially observed nonlinear diffusion models using the Metropolis-Hastings algorithm. *Biometrika*, 88:603–621, 2001. doi: 10.1093/biomet/88.3.603.

Kevin Tan, Aaron J. Abkemeier, Kunyang He, Giles Hooker, and Edward L. Ionides. Automatic differentiation for inference on partially observed Markov process models. *arXiv:2407.03085v2*, 2026. doi: 10.48550/arXiv.2407.03085.

Nico M. Temme. Asymptotic inversion of incomplete gamma functions. *Mathematics of Computation*, 58(198):755–764, 1992. doi: 10.1090/S0025-5718-1992-1122079-8.

Li Wen, Yi Yin, Qiong Li, Zhihang Peng, and Daihai He. Modeling the co-circulation of influenza and COVID-19 in Hong Kong, China. *Advances in Continuous and Discrete Models*, 2024(32(2024)):1–9, 2024. doi: 10.1186/s13662-024-03830-7.

Jesse Wheeler and Edward L Ionides. Revisiting inference for ARMA models: Improved fits and superior confidence intervals. *PloS One*, 20(10):e0333993, 2025. doi: 10.1371/journal.pone.0333993.

Jesse Wheeler, Aaron J. Abkemeier, and Edward L. Ionides. Iterating marginalized Bayes maps for likelihood maximization with application to nonlinear panel models. *arXiv:2511.17438*, 2025. doi: 10.48550/arXiv.2511.17438.

Appendix A. Full benchmark results

The four benchmark models stress different parts of the particle filter, which helps locate performance bottlenecks.

Cholera, Dhaka. The cholera model of King et al. (2008) has 600 monthly observations with 20 Euler steps between each, and each step updates a stochastic differential equation with one normal draw. The process model is cheap, but there are many steps between resamples, so this model isolates the per-step overhead inside the process model.

Measles, London. The measles model of He et al. (2010) has weekly case reports from 1950 to 1963 and takes a daily Euler step, interpolating covariates and drawing binomial, gamma, and Poisson random variables at every step. It has a relatively expensive process model, so it isolates how the GPU handles heavy computation per particle, whilte testing the fast samplers in `pypomp.random` (Appendix D).

Measles, 4-town panel. The panel model fits the same measles model to London, Halesworth, Hastings, and Cardiff jointly, with some parameters shared across towns and the rest specific to each town. MPIF (Wheeler et al., 2025) filters the towns sequentially within each iteration, so an iteration costs about four single-town iterations with the same number of particles in flight. This model tests the panel methods used in Appendix E. Its ratio of R pomp CPU time to GPU time is lower than for the London model because R runs it relatively fast, likely because the random variate generators in R can take a cheaper route for the smaller parameter values of the smaller towns.

S&P 500. The stochastic volatility model of Bretó (2014) has 3,523 daily log returns and takes one step of its process model per observation, with a single normal draw. Each step between resamples is very cheap, so fixed per-observation costs such as resampling can dominate.

Table 1 benchmarks Pypomp 1.0.6--1.3.0rc1 against pomp 6.3 and panelPomp 1.7.0.0. R pomp and Pypomp run the same workload in a given benchmark. Each benchmark runs 36 IF2 searches from the same starting points and then runs replicate particle filters at each search's IF2 estimate, with as many particles per filter as per search. The cholera and both measles benchmarks use 5,000 particles, 100 IF2 iterations, and 36 particle filter replicates, and the S&P 500 benchmark uses 1,000 particles, 200 iterations, and 24 replicates.

Fitting the measles model of He et al. (2010) to all 1,422 towns and cities for 1950–1963 took one GPU 38 hours, including likelihood evaluation, versus an estimated 4.8 months for R on a 36-core node, extrapolating linearly from panelPomp's cost on the 4-town panel (Appendix E). Thus, Pypomp enables a scientific analysis that was previously considered unfeasible (Becker and Grenfell, 2017).

Table 2 extends Table 1 with particle filter times for Pypomp on CPU, using the same 36 cores used for R pomp. Pypomp on CPU was 2.5 to 5.6 times faster than R pomp, and moving Pypomp to the GPU makes it another 3.8 to 33 times faster. These workloads are sized for R and too small to saturate the GPU, so the hardware and total ratios understate what the GPU can do. At its peak throughput in the scaling tests (Appendix C), one GPU runs IF2 92 to 232 times as fast as R on the 36-core node.

Table 2: Wall-clock time in seconds for IF2 and for particle filter (PF) evaluation, with the workloads described above. Pypomp IF2 and PF times include JIT compilation. Software ratio is the pomp (or panelPomp) time divided by the Pypomp time on the same 36 cores, hardware ratio is the Pypomp CPU time divided by the Pypomp GPU time, and total is their product, the pomp time divided by the Pypomp GPU time. These workloads do not saturate the GPU, so the hardware and total ratios understate its throughput; peak cores matched in Table 1 gives the GPU's highest measured IF2 throughput. The 4-town panel has no Pypomp CPU run.

		`pomp`	Pypomp		Ratio		
Model	Task	36 cores	36 cores	1 GPU	Software	Hardware	Total
Cholera, Dhaka	IF2	1,609	390	28.3	4.1×	14×	57×
	PF	380	67.8	8.2	5.6×	8.3×	46×
Measles, London	IF2	2,923	1,071	32.2	2.7×	33×	91×
	PF	895	302	14.5	3.0×	21×	62×
Measles, 4-town panel	IF2	8,262	—	98.2	—	—	84×
	PF	2,252	—	33.7	—	—	67×
S&P 500	IF2	643	260	34.8	2.5×	7.5×	19×
	PF	32.0	6.9	1.8	4.7×	3.8×	18×

Appendix B. Comparison with other particle filter libraries

Table 3 times the particle filter workloads of Table 2 in other Python particle filter libraries, with each model written as a user of that library would write it. Every library runs the bootstrap particle filter with the same number of filters and particles as in Table 2. The GPU libraries ran on one RTX 3090, a slower GPU than the one used in Table 1, and with a different Pypomp release, so the Pypomp times in Table 3 are not comparable with those in Table 2. We left out Dynamax (Linderman et al., 2025), since its only particle filter is a Rao–Blackwellized filter for switching linear Gaussian models.

On the cholera model, `pfjax` and `cuthbert` took 1.0 to 1.2 times as long as Pypomp, so the three JAX libraries performed about the same. On the measles models, `pfjax` and `cuthbert` were 23 to 26 times slower than Pypomp for London, and neither finished the 4-town panel within an hour. Most of this gap comes from the `jax.random` rejection samplers, which these libraries use for the binomial, gamma, and Poisson draws of the measles model (Appendix D). The PyTorch library `pydpf` took 2.2 to 5.4 times as long as Pypomp on the cholera and measles models.

The S&P 500 model is the exception. Its steps are cheap, so the cost of running each parameter set's replicate filters one after another dominates, and Pypomp 1.2.0 was the slowest of the GPU libraries, taking 5.4 seconds against 2.7 seconds for `cuthbert`. The development version of Pypomp runs these replicates in parallel and takes 2.2 seconds, faster than every other GPU library.

The CPU-only libraries `particles` and `pypfilt` were timed on one core, and we scaled each time to 36 cores assuming perfect scaling, which flatters them. Under this assumption,

Table 3: Wall-clock time in seconds for the particle filter benchmarks of Table 2 in Pypomp 1.2.0, `pfjax` 0.0.3rc1 and `cuthbert` 0.1.1 (JAX), `pydpf` 1.2.3 (PyTorch), and `particles` 0.3 and `pypfilt` 0.8.7 (NumPy). GPU times are from one RTX 3090 and include JIT compilation. For the measles models, `pfjax` and `cuthbert` draw with the `jax.random` samplers (Appendix D). †`particles` and `pypfilt` run only on CPUs; their times are the time for one filter on one core, multiplied by the number of filters and divided by 36. *`panelPomp` for the 4-town panel, as in Table 2.

	1 GPU				36 CPU cores		
Model	Pypomp	`pfjax`	`cuthbert`	`pydpf`	`particles`†	`pypfilt`†	`pomp`*
Cholera, Dhaka	12.7	15.0	12.8	68.3	203	429	380
Measles, London	53.8	1,400	1,224	152	450	897	895
Measles, 4-town panel	145	>3,600	>3,600	321	2,166	4,393	2,252
S&P 500	5.4	4.6	2.7	3.3	16.8	301	32.0

`particles` took 0.5 to 1.0 times the measured pomp time, and `pypfilt` took 1.0 to 9.4 times the pomp time.

Appendix C. GPU scaling tests

The benchmarks in Table 1 and Table 2 use the R configuration, which does not give the GPU enough work to reach its highest throughput for every model. The scaling tests in quant ran IF2 on one RTX PRO 6000 Blackwell Server Edition GPU with more particles in flight, added in two ways: more searches of 1,000 particles each, or more particles for each of a fixed number of searches. Each configuration ran in its own process and ran a short IF2 twice, so that the second run excludes JIT compilation. We project each configuration's time to the benchmark's number of IF2 iterations, including compilation, and divide the benchmark's R time by it to get the number of CPU cores running R that one GPU matches. This assumes that R's time grows linearly with the number of particles and searches. The highest of these values is the Peak column of Table 1. JAX's memory preallocation was turned off, so the peak GPU memory is the memory each configuration used. No scaling test used more than 1.6 GB of GPU memory, the peak for the cholera model with 2.4 million particles.

Figure 1 shows the results. For every model, throughput peaks at between 1.0 and 2.0 million particles and falls or plateaus at larger sizes. The peak is 1.6 times the benchmark for the cholera model, 1.7 times for the London measles model, 1.3 times for the 4-town panel, and 12.5 times for the S&P 500 model. The S&P 500 model gains the most because its benchmark, with cheap steps and few particles, leaves the GPU mostly idle. For this model, adding searches beats adding particles per search at a given total number of particles by an average of 7%. Consequently, there is not a large difference in speed between adding searches or particles per search on the GPU in terms of scaling with total particles.

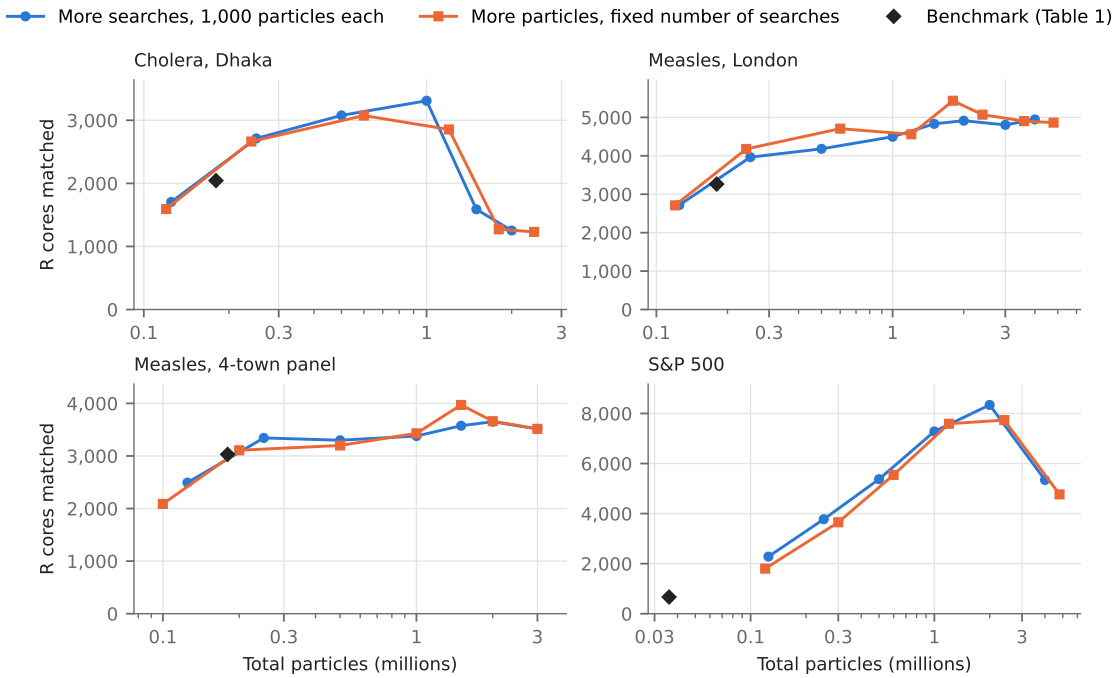


Figure 1: IF2 throughput on one GPU, as the number of CPU cores running R that the GPU matches, against the total number of particles in flight. Blue adds searches of 1,000 particles each; orange adds particles to each of 100 or 120 searches. Each configuration ran 3 or 10 IF2 iterations, projected to the benchmark's number. The x-axes use a log scale.

Table 4: Time in milliseconds for 1,000,000 draws from `pypomp.random` and `jax.random`, on one GPU and on one CPU core, with parameters spread across the values where the samplers switch algorithms. Ratio is the `jax.random` time divided by the `pypomp.random` time. `jax.random` has no negative binomial sampler. Max W_1/σ is the largest Wasserstein distance between `pypomp.random` draws and draws from `jax.random` (or `scipy.stats` for the negative binomial) over a grid of parameter values, relative to the standard deviation.

	GPU			CPU			
Distribution	`pypomp`	`jax`	Ratio	`pypomp`	`jax`	Ratio	Max W_1/σ
Poisson	0.03	1.05	35×	37.17	1,321	36×	0.010
Binomial	0.02	1.07	44×	24.56	1,349	55×	0.010
Gamma	0.03	0.76	23×	118	3,046	26×	0.011
Negative binomial	0.07	—	—	187	—	—	0.009

Appendix D. Random variable samplers

`jax.random` draws Poisson, binomial, and gamma random variables with rejection samplers, which loop a random number of times per draw. This suits GPUs poorly, since threads that run together wait for the slowest one. `pypomp.random` instead provides approximate inverse-CDF samplers, which avoid rejection loops. Table 4 compares the two on 1,000,000 draws per call, with parameters spread across the values where each sampler switches algorithms, averaged over 20 calls. The approximation error is small: over a grid of parameter values, the Wasserstein distance between 1,000,000 draws from each `pypomp.random` sampler and from a reference sampler is at most 1.1% of the reference standard deviation. In the London measles benchmark, the `jax.random` samplers made IF2 46 times slower and particle filtering 36 times slower on the GPU.

Appendix E. England and Wales measles fit

We ran 36 MPIF searches, each with 10,000 particles for 200 iterations, and evaluated each endpoint with 36 particle filter replicates of 5,000 particles. We then combined the best estimate for each town across searches and evaluated the result with another 36 replicates. Table 5 gives the time for each stage on one GPU with Pypomp 1.0.8.

For comparison, we estimated how long the same computation would take with panelPomp on a 36-core CPU node. We measured panelPomp's cost per particle, per IF2 iteration, and per town on the 4-town panel in Table 2, which covers the same period with the same daily time step, and scaled it linearly to the full computation. We assume panelPomp fits the towns jointly with MPIF, as Pypomp did, so each search runs on one core and does not parallelize across units – this would be necessary in the general case when using shared parameters. By this estimate, R would need about 4.8 months, 92 times longer than the GPU, and about 14.3 years on a single core, so the GPU delivered the throughput of about 3,300 CPU cores running R. The wall clock time can be reduced further by spreading the replication over multiple GPUs; for example, a run with the same settings but only 12

Table 5: Wall-clock time for fitting the measles model to all towns in England and Wales. Pypomp times were measured on one GPU. R panelPomp times are estimates for a 36-core CPU node, extrapolated from the 4-town panel in Table 2.

Stage	Pypomp (1 GPU)	R `panelPomp`, 36 cores (estimated)
IF2	35.5 hours	136.0 days
PF, all search endpoints	2.5 hours	9.3 days
PF, combined estimate	6 minutes	6.2 hours
Total	38.1 hours	145.5 days

searches took one GPU 15 hours, so three GPUs running 12 searches each would finish in about 15 hours instead of 38.